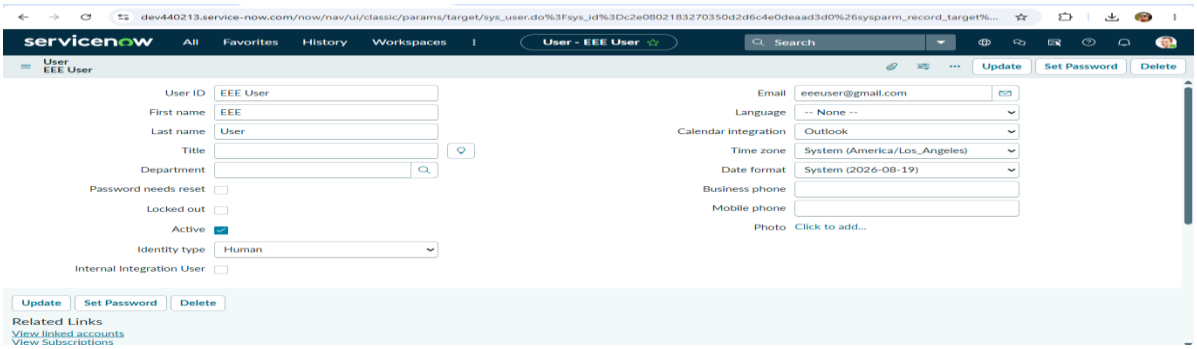
SCRIPT-CONTROLLED ACL Restrict Record Access Based on Field Value (ServiceNow)

Phase 5 - Project Development

This phase covers the actual, hands-on implementation carried out on the ServiceNow instance, following the milestones defined in the planning phase.

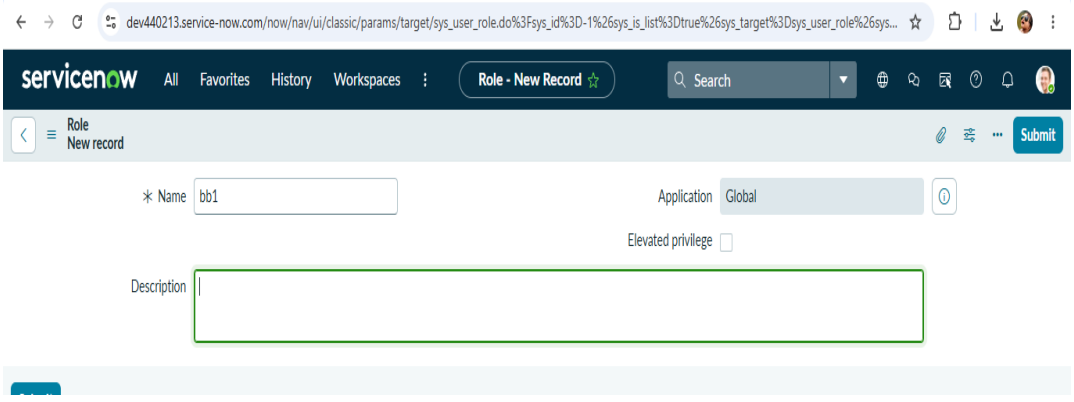
5.1 Milestone 1 — Creation of Users and Roles

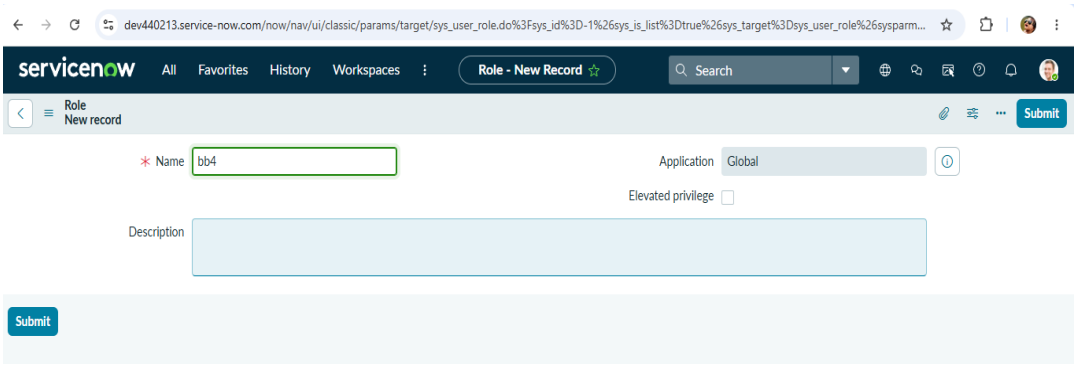
5.1.1 User Creation



1. Log in to the ServiceNow instance with admin access.
2. Navigate to User Administration → Users and click New.
3. Enter User ID: EEEUser, First name: EEE, Last name: User, Email: eeeuser@gmail.com.
4. Save the record.

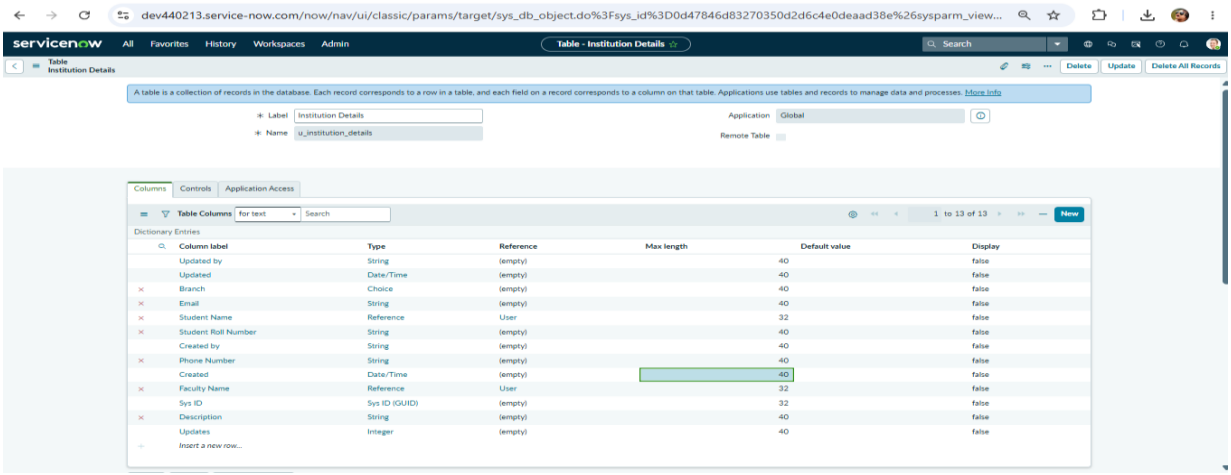
5.1.2 Role Creation





5. Navigate to User Administration → Roles and click New.
6. Enter Name: bb1 and a brief Description of the role's purpose, then click Submit.
7. Repeat the same procedure to create roles bb2, bb3 and bb4.
8. Assign all four roles to the EEE User record so the account can be used to test every operation.

5.2 Milestone 2 — Table Creation



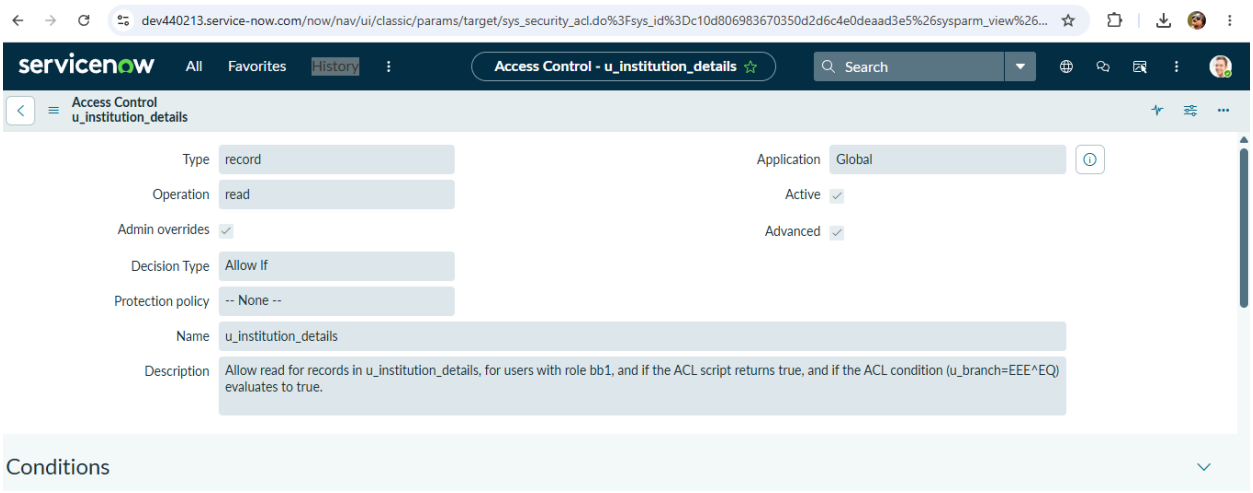
9. In the Application Navigator, search for System Definition → Tables and click New.
10. Set Label: Institution Details, Name: u_institution_details, Extends: false, then click Submit.

The following fields were then added to the table:

Field	Type
Student Roll Number	Auto Number
Student Name	Reference – User
Faculty Name	Reference – User
Branch	Choice (ECE, EEE, CSE)
Email	String
Phone Number	String
Description	Multi-line String

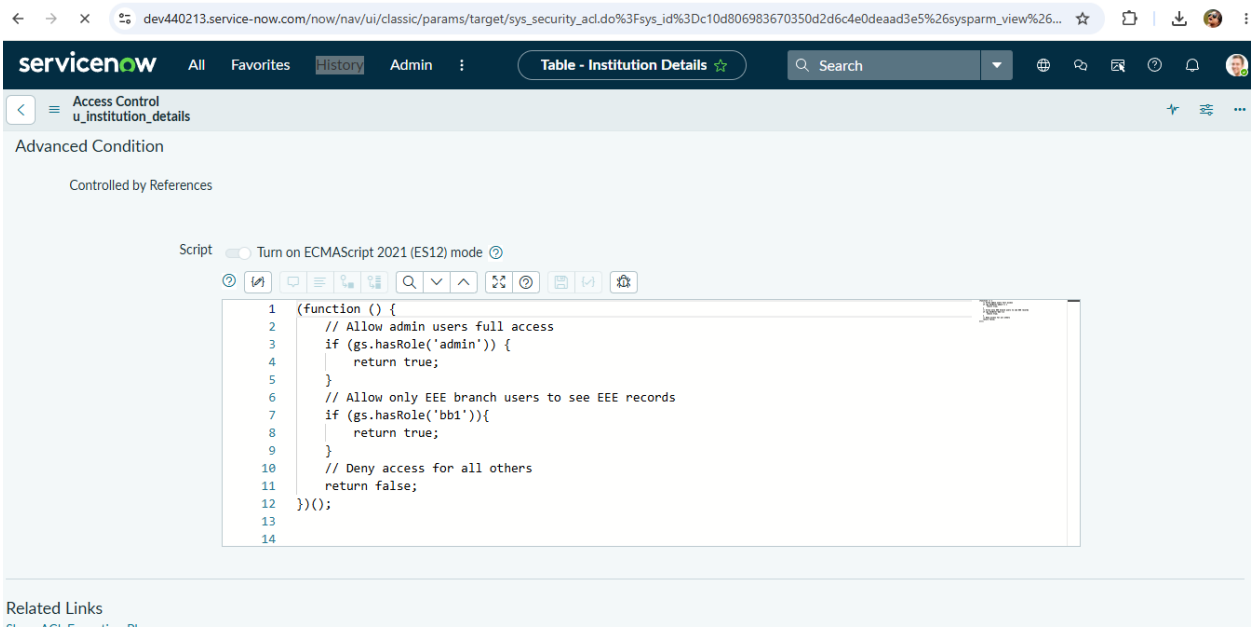
Multiple sample records were then created in the Institution Details list with different Branch values (ECE, EEE, CSE) so that the ACL behaviour could later be verified across branches.

5.3 Milestone 3 — Read ACL (Script-Controlled)



Navigate to System Security → Access Control (ACL).

11. Elevate to the security_admin role (required to create or modify ACLs).
12. Click New to create a new ACL and set Type: record, Operation: read, Name: u_institution_details, Active: true, Advanced: true.
13. In the Requires role related list, add the custom role bb1.
14. Add a Data condition: Branch is EEE.
15. In the Script field, add the advanced server-side script shown in the Design phase (admin override, then bb1 role check, then deny by default).
16. Click Submit to save the ACL.



5.4 Milestone 4 — Create ACL

17. Click New to create a new ACL (Create) with Type: record, Operation: create, Name: u_institution_details, Active: true.
18. In the Requires role related list, add role bb2. No data condition is required for this operation.
19. Save the ACL record.

5.5 Milestone 5 — Write ACL

20. Click New to create a new ACL (Write) with Type: record, Operation: write, Name: u_institution_details, Active: true.
21. In the Requires role related list, add role bb3. No data condition is required.
22. Save the ACL record.

5.6 Milestone 6 — Delete ACL

23. Click New to create a new ACL (Delete) with Type: record, Operation: delete, Name: u_institution_details, Active: true.
24. In the Requires role related list, add role bb4. No data condition is required.
25. Save the ACL record.

5.7 Summary of Developed Components

Component	Count	Details
Custom Table	1	u_institution_details (7 fields)
Custom Roles	4	bb1, bb2, bb3, bb4
ACL Rules	4	Read (scripted), Create, Write, Delete
Test Users	2	EEE User (branch user), System Administrator

5.8 Sample Data Used

Roll No.	Student Name	Faculty Name	Branch
INS0001001	Yaswanth Nikku	Abraham Lincoln	CSE
INS0001004	Prashanth Reddy	Abel Tuter	EEE
INS0001005	Srinu Nandipalli	ITIL User	ECE
INS0001006	Praneeth	ITIL User	ECE
INS0001007	Guna Sai	Abel Tuter	EEE
INS0001008	Abraham Lincoln	Malla Sravan kumar	CSE

5.9 Code Walk through

Line by line, the Read ACL script first wraps the logic in an immediately-invoked anonymous function to keep the ACL's scripting scope clean. The first conditional, gs.hasRole('admin'), calls ServiceNow's GlideSystem API to check whether the currently logged-in user holds the administrator role; if so the function returns true immediately and no further checks are made. The second conditional performs the same check for the bb1 role, which — combined with the ACL's own Data Condition of Branch is EEE — means this branch only returns true for a bb1 user when the specific record being evaluated also belongs to the EEE branch. If neither condition is met, the function falls through to the final 'return false', denying access. The Create, Write and Delete ACLs are simpler: they rely solely on the built-in Requires-role check (bb2, bb3 and bb4 respectively) with no advanced script, since those operations do not need to be filtered by branch in this project — only Read needed a per-record, field-value-based restriction.